

Optimized tool selection! I'm cleaning up the SQL interview notes so they read properly and consistently, starting by correcting the wording and formatting in the current document.

Real SQL Interview Questions for 10–20 LPA Roles

1. Write a query to find employees with the second highest salary in a table.

Use a subquery to find the highest salary first, then select the maximum value below it.

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

LinkedIn@AshishZope

Explanation:

- The inner query finds the maximum salary.
- The outer query finds the highest salary lower than that value.
- This gives the second-highest salary.

Tip:

- If you want to handle ties or fetch multiple employees with the same second-highest salary, use `DENSE_RANK()` with a CTE.

2. Explain the difference between INNER JOIN and OUTER JOIN with examples.

INNER JOIN

Returns only matching rows from both tables.

```
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d
ON e.department_id = d.department_id;
```

This returns only employees who belong to a valid department.

LEFT OUTER JOIN

Returns all rows from the left table and the matching rows from the right table. If there is no match, it returns **NULL**.

```
SELECT e.name, d.department_name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id;
```

This returns all employees, along with department details when available.

RIGHT OUTER JOIN

Returns all rows from the right table and matching rows from the left table.

FULL OUTER JOIN

Returns all rows from both tables, with matches where available and **NULL** where not.

Key difference:

- **INNER JOIN** = common data only
- **OUTER JOIN** = all data from one or both sides, with **NULL** for missing matches

3. Write a query to fetch the second-highest salary from an employee table.

Option 1: Using DISTINCT, ORDER BY, and LIMIT (MySQL/PostgreSQL)

LinkedIn@AshishZope

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Option 2: Using a subquery (generic SQL)

LinkedIn@AshishZope

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Explanation:

- The subquery gets the highest salary.
- The outer query gets the largest value below it.

4. How do you use GROUP BY and HAVING together? Provide an example.

Use **GROUP BY** to group rows and **HAVING** to filter aggregated results.

DATA ENGINEER

LinkedIn@AshishZope

```
SELECT department_id, COUNT(*) AS emp_count
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Explanation:

- Rows are grouped by **department_id**
- The count is calculated for each department
- Only departments with more than 5 employees are returned

Note:

- **WHERE** filters rows before grouping
- **HAVING** filters after aggregation

5. Write a query to find employees earning more than their managers.

Assume the table has:

- emp_id
- name
- salary
- manager_id

```
SELECT e.name AS employee_name,  
       e.salary,  
       m.name AS manager_name,  
       m.salary AS manager_salary  
FROM employees e  
JOIN employees m  
  ON e.manager_id = m.emp_id  
WHERE e.salary > m.salary;
```

LinkedIn@AshishZope

Explanation:

- This is a self-join
- One instance of the employees table is treated as the employee
- The other instance is treated as the manager

6. What is a window function in SQL? Provide examples of ROW_NUMBER and RANK.

A window function performs a calculation across a set of related rows without collapsing them like GROUP BY does.

Syntax:

```
FUNCTION_NAME() OVER (PARTITION BY column ORDER BY column)
```

LinkedIn@AshishZope

Example: ROW_NUMBER()

LinkedIn@AshishZope

```
SELECT name, department, salary,  
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS  
row_num  
FROM employees;
```

This assigns a unique number to each employee within the same department based on salary.

Example: RANK()

LinkedIn@AshishZope

```
SELECT name, department, salary,  
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_num  
FROM employees;
```

This gives the same rank to employees with the same salary, but skips the next rank.

Example:

- If two employees tie at rank 1, the next employee gets rank 3

7. Write a query to fetch the top 3 performing products based on sales.

Assume the table `sales_data` has:

- `product_id`
- `product_name`
- `total_sales`

LinkedIn@AshishZope

```
SELECT product_id, product_name, total_sales  
FROM sales_data  
ORDER BY total_sales DESC  
LIMIT 3;
```

Alternate version using `RANK()`:

LinkedIn@AshishZope

```
SELECT product_id, product_name, total_sales
FROM (
    SELECT *,
           RANK() OVER (ORDER BY total_sales DESC) AS rank_num
    FROM sales_data
) ranked_sales
WHERE rank_num <= 3;
```

8. Explain the difference between UNION and UNION ALL.

Feature	UNION	UNION ALL
Duplicates	Removes duplicates	Keeps duplicates
Performance	Slower	Faster
Use Case	Distinct results	Include all rows

Example:

LinkedIn@AshishZope

```
SELECT city
FROM customers
UNION
SELECT city
FROM vendors;
```

This returns unique city names.

LinkedIn@AshishZope

```
SELECT city
FROM customers
UNION ALL
SELECT city
FROM vendors;
```

This returns every city including duplicates.

9. How do you use a CASE statement in SQL? Provide an example.

CASE is used for conditional logic inside a query, similar to **IF...ELSE**.

```
SELECT name, salary,
       CASE
         WHEN salary >= 100000 THEN 'High'
         WHEN salary >= 50000  THEN 'Medium'
         ELSE 'Low'
       END AS salary_category
FROM employees;
```

LinkedIn@AshishZope

This categorizes employees based on their salary.

10. Write a query to calculate the cumulative sum of sales.

Assume the **sales** table has:

- **order_date**
- **product_id**
- **sales_amount**

```
SELECT order_date,
       product_id,
       sales_amount,
       SUM(sales_amount) OVER (
         PARTITION BY product_id
         ORDER BY order_date
       ) AS cumulative_sales
FROM sales;
```

LinkedIn@AshishZope

Explanation:

- **PARTITION BY product_id** calculates the running total per product
- **ORDER BY order_date** ensures the cumulative total follows chronological order

11. What is a CTE (Common Table Expression), and how is it used?

A CTE is a temporary named result set that can be referenced within a SQL query. It improves readability and makes complex queries easier to manage.

Syntax:

```
WITH cte_name AS (  
    SELECT ...  
)  
SELECT *  
FROM cte_name;
```

LinkedIn@AshishZope

Example:

```
WITH HighEarnings AS (  
    SELECT emp_id, name, salary  
    FROM employees  
    WHERE salary > 100000  
)  
SELECT *  
FROM HighEarnings;
```

LinkedIn@AshishZope

Benefits:

- Improves readability
- Simplifies complex subqueries
- Supports recursion for hierarchical data

12. Write a query to identify customers who have made transactions above \$5,000 multiple times.

Assume the `transactions` table has:

- `customer_id`
- `transaction_amount`


```
SELECT customer_id, COUNT(*) AS high_value_txns
FROM transactions
WHERE transaction_amount > 5000
GROUP BY customer_id
HAVING COUNT(*) > 1;
```

Explanation:

- Filters transactions above 5000
- Groups them by customer
- Returns only customers with more than one such transaction

13. Explain the difference between DELETE and TRUNCATE.

Feature	DELETE	TRUNCATE
Removes rows	Yes	Yes
WHERE clause	Yes	No
Logging	More logging	Minimal logging
Speed	Slower	Faster
Rollback	Yes, if in a transaction	Can be rolled back in some DBs
Identity reset	No	Yes, in most DBs
Best use case	Partial deletion, audit-friendly tasks	Full table cleanup

DELETE is row-specific and can be filtered. **TRUNCATE** is designed for removing all rows quickly.

14. How do you optimize SQL queries for better performance?

Some common optimization techniques are:

1. Select only required columns

```
SELECT * FROM orders;    -- inefficient
SELECT order_id, customer_id FROM orders;    -- better
```

LinkedIn@AshishZope

2. Create proper indexes on columns used in:

- WHERE
- JOIN
- ORDER BY
- GROUP BY

3. Avoid using functions on indexed columns

```
WHERE YEAR(order_date) = 2024    -- slower
```

LinkedIn@AshishZope

Better:

```
WHERE order_date BETWEEN '2024-01-01' AND '2024-12-31'
```

LinkedIn@AshishZope

4. Prefer EXISTS over IN for subqueries in many cases:

```
SELECT name
FROM customers c
WHERE EXISTS (
    SELECT 1
    FROM orders o
    WHERE o.customer_id = c.customer_id
);
```

LinkedIn@AshishZope

5. Reduce unnecessary joins and nested subqueries

6. Use the correct data types

7. Analyze execution plans using EXPLAIN or EXPLAIN ANALYZE

15. Write a query to find all customers who have not made any purchases in the last 6 months.

Assume:

- `customers(customer_id, name)`
- `transactions(customer_id, transaction_date)`

LinkedIn@AshishZope

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN transactions t
  ON c.customer_id = t.customer_id
 AND t.transaction_date >= CURRENT_DATE - INTERVAL '6 months'
WHERE t.customer_id IS NULL;
```

Explanation:

- The `LEFT JOIN` keeps all customers
- The condition filters only recent purchases
- `NULL` values indicate customers with no purchases in the last 6 months

16. How do you handle NULL values in SQL? Provide examples.

`NULL` means missing or unknown data.

1. Check for NULL values

LinkedIn@AshishZope

```
SELECT *
FROM employees
WHERE manager_id IS NULL;
```

2. Replace NULL with a value

LinkedIn@AshishZope

```
SELECT name, COALESCE(phone_number, 'Not Provided') AS contact
FROM customers;
```

3. Aggregate functions ignore NULLs by default

LinkedIn@AshishZope

```
SELECT AVG(salary)
FROM employees;
```

4. Use CASE to handle NULLs

LinkedIn@AshishZope

```
SELECT name,
       CASE
         WHEN salary IS NULL THEN 'Unknown'
         ELSE 'Known'
       END AS salary_status
FROM employees;
```

17. Write a query to transpose rows into columns.

Assume a table `sales`:

- `region`
- `month`
- `sales_amount`

Using `CASE`:

LinkedIn@AshishZope

```
SELECT region,
       SUM(CASE WHEN month = 'Jan' THEN sales_amount ELSE 0 END) AS Jan,
       SUM(CASE WHEN month = 'Feb' THEN sales_amount ELSE 0 END) AS Feb,
       SUM(CASE WHEN month = 'Mar' THEN sales_amount ELSE 0 END) AS Mar
FROM sales
GROUP BY region;
```

Using **PIVOT** syntax in SQL Server/Oracle:

LinkedIn@AshishZope

```
SELECT region, [Jan], [Feb], [Mar]
FROM (
    SELECT region, month, sales_amount
    FROM sales
) AS src
PIVOT (
    SUM(sales_amount)
    FOR month IN ([Jan], [Feb], [Mar])
) AS p;
```

18. Explain indexing and how it improves query performance.

An index is a data structure that helps search and retrieve data faster from a database table. It reduces the number of rows scanned.

Without index

- Database scans the full table
- Query performance is slow

With index

- Database uses the index to locate rows quickly
- Query performance improves significantly

Example:

LinkedIn@AshishZope

```
CREATE INDEX idx_customer_id ON transactions(customer_id);
```

This helps:

```
SELECT *  
FROM transactions  
WHERE customer_id = 101;
```

Important notes:

- Too many indexes can slow writes (**INSERT**, **UPDATE**)
- Avoid indexing low-cardinality columns like **gender**
- Use composite indexes when multiple columns are often queried together

19. Write a query to fetch the maximum transaction amount for each customer.

Assume:

- **transactions(customer_id, transaction_id, amount)**

```
SELECT customer_id, MAX(amount) AS max_transaction  
FROM transactions  
GROUP BY customer_id;
```

Explanation:

- Rows are grouped by customer
- **MAX(amount)** returns the highest transaction for each customer

20. What is a self-join, and how is it used?

A self-join joins a table to itself. It is useful when rows in the same table relate to other rows in the same table.

Example: Employees and managers

Assume:

emp_id	name	manager_id
1	Alice	NULL
2	Bob	1
3	Carol	1
4	David	2

LinkedIn@AshishZope

Query:

```
SELECT e.name AS employee_name,  
       m.name AS manager_name  
FROM employees e  
LEFT JOIN employees m  
  ON e.manager_id = m.emp_id;
```

LinkedIn@AshishZope

Explanation:

- **e** is the employee
- **m** is the manager
- The join matches manager IDs with employee IDs

Data Analysis / Scenario-Based Questions

21. How would you design a database to store credit card transaction data?

A well-designed schema should be normalized, secure, and scalable.

Suggested tables:

1. Customers

- customer_id
- name
- email
- phone

- `address`

2. `Cards`

- `card_id`
- `customer_id`
- `card_number`
- `card_type`
- `status`
- `issued_date`

3. `Merchants`

- `merchant_id`
- `name`
- `category`
- `location`

4. `Transactions`

- `transaction_id`
- `card_id`
- `merchant_id`
- `transaction_date`
- `amount`
- `currency`
- `status`
- `location`

Best practices:

- Mask or encrypt card numbers
- Use tokenization for sensitive data
- Add indexes on `card_id`, `merchant_id`, and `transaction_date`
- Partition large transaction tables by date

22. Write a query to identify the most profitable regions based on transaction data.

Assume:

- `transactions(transaction_id, customer_id, amount, region, transaction_date)`

```
SELECT region, SUM(amount) AS total_revenue
FROM transactions
GROUP BY region
ORDER BY total_revenue DESC
LIMIT 3;
```

LinkedIn@AshishZope

Explanation:

- It aggregates transaction amounts by region
- Sorts by total revenue
- Returns the top three regions

23. How would you analyze customer churn using SQL?

A churned customer is typically one who has not transacted in a defined period, such as the last 6 months.

Example schema:

- `customers(customer_id, name, signup_date)`
- `transactions(customer_id, transaction_date, amount)`

Query to identify churned customers:

```
SELECT c.customer_id, c.name
FROM customers c
LEFT JOIN transactions t
  ON c.customer_id = t.customer_id
  AND t.transaction_date >= CURRENT_DATE - INTERVAL '6 months'
WHERE t.transaction_id IS NULL;
```

You can then calculate:

- Churn rate
- Monthly churn trend
- Average spend of churned vs active customers

24. Explain the difference between OLAP and OLTP databases.

Feature	OLTP	OLAP
Purpose	Transaction processing	Data analysis
Operations	Insert, Update, Delete	Read-heavy analysis queries
Schema	Normalized	Denormalized
Speed	Fast for single-row operations	Fast for large analytical queries
Typical users	Clerks, DBAs	Analysts, data scientists
Example systems	Banking, ecommerce	BI dashboards, reporting

Summary:

- OLTP = operational and real-time
- OLAP = analytical and historical

25. How would you determine the Average Revenue Per User (ARPU) from transaction data?

ARPU is:

$$ARPU = \frac{\text{Total Revenue}}{\text{Total Number of Users}}$$

LinkedIn@AshishZope

```
SELECT
    SUM(amount) * 1.0 / COUNT(DISTINCT customer_id) AS ARPU
FROM transactions;
```

Explanation:

- `SUM(amount)` gives total revenue
- `COUNT(DISTINCT customer_id)` gives the number of users
- Multiplying by `1.0` ensures floating-point division

Monthly ARPU example:

LinkedIn@AshishZope

```
SELECT
    DATE_TRUNC('month', transaction_date) AS month,
    SUM(amount) * 1.0 / COUNT(DISTINCT customer_id) AS monthly_arpu
FROM transactions
GROUP BY DATE_TRUNC('month', transaction_date)
ORDER BY month;
```

DATA ENGINEER

26. Describe a scenario where you would use a LEFT JOIN instead of an INNER JOIN.

Use a `LEFT JOIN` when you want all rows from the left side, even if there is no match in the right table.

Example:

- List all customers and their transactions, even if some customers have never purchased anything.

LinkedIn@AshishZope

```
SELECT c.customer_id, c.name, t.transaction_id, t.amount
FROM customers c
LEFT JOIN transactions t
ON c.customer_id = t.customer_id;
```

Why use **LEFT JOIN**:

- It includes all customers
- Customers with no transactions will have **NULL** values in transaction columns
- An **INNER JOIN** would exclude them

27. Write a query to calculate YoY (Year-over-Year) growth for a set of transactions.

Assume a table:

- **transactions(customer_id, transaction_date, amount)**

First, calculate yearly revenue:

LinkedIn@AshishZope

```
SELECT
    EXTRACT(YEAR FROM transaction_date) AS year,
    SUM(amount) AS total_revenue
FROM transactions
GROUP BY EXTRACT(YEAR FROM transaction_date);
```

Then calculate YoY growth using a CTE and a self-join:

```
WITH yearly_revenue AS (  
    SELECT  
        EXTRACT(YEAR FROM transaction_date) AS year,  
        SUM(amount) AS total_revenue  
    FROM transactions  
    GROUP BY EXTRACT(YEAR FROM transaction_date)  
)  
SELECT  
    curr.year AS current_year,  
    curr.total_revenue,  
    prev.total_revenue AS previous_year_revenue,  
    ROUND(((curr.total_revenue - prev.total_revenue) / prev.total_revenue) *  
100, 2) AS yoy_growth_percent  
FROM yearly_revenue curr  
LEFT JOIN yearly_revenue prev  
    ON curr.year = prev.year + 1;
```

This compares each year to the previous year and calculates the percentage change.

28. How would you implement fraud detection using transactional data?

Fraud detection usually combines:

- Rule-based detection
- Pattern recognition
- Statistical anomaly detection

Examples of rules:

- Transactions much larger than a customer's average
- Multiple transactions from the same user in a short time
- Location mismatches
- Same card used by different customers or devices

Example query – find unusually large transactions:

```
WITH avg_txn AS (  
    SELECT customer_id, AVG(amount) AS avg_amount  
    FROM transactions  
    GROUP BY customer_id  
)  
SELECT t.*  
FROM transactions t  
JOIN avg_txn a  
    ON t.customer_id = a.customer_id  
WHERE t.amount > 3 * a.avg_amount;
```

This flags transactions that are more than 3 times the customer's average.

29. Write a query to find customers who have used more than 2 credit cards for transactions in a given month.

Assume:

- `transactions(customer_id, card_id, transaction_date)`

```
SELECT customer_id,  
    TO_CHAR(transaction_date, 'YYYY-MM') AS txn_month,  
    COUNT(DISTINCT card_id) AS cards_used  
FROM transactions  
GROUP BY customer_id, TO_CHAR(transaction_date, 'YYYY-MM')  
HAVING COUNT(DISTINCT card_id) > 2;
```

Explanation:

- Groups data by customer and month
- Counts unique cards used
- Returns only customers using more than 2 cards in the same month

30. How would you approach a business problem where you need to analyze the spending patterns of premium customers?

A structured approach is:

Step 1: Understand the objective

Clarify what “spending pattern” means:

- Frequency
- Amount
- Category
- Channel
- Timing

Define premium customers:

- Card tier
- Monthly spend threshold
- Credit score
- Membership level

Step 2: Collect relevant data

Typical tables:

- Customers
- Transactions
- Cards
- Merchants

Step 3: Clean and prepare data

- Handle missing values
- Remove outliers
- Filter premium customers
- Enrich data such as merchant category or region

Step 4: Perform exploratory analysis

Look at:

- Average monthly spend
- Spending trends over time
- Category-wise purchases
- Geography-wise behavior
- Seasonal effects

Step 5: Segment the audience

Examples:

- High spenders
- Frequent spenders
- Category loyalists
- Low-engagement premium customers

Step 6: Recommend actions

- Personalized offers
- Better retention strategies
- Upgrade suggestions
- Category-based rewards

Example SQL:

```
SELECT customer_id,
       DATE_TRUNC('month', transaction_date) AS txn_month,
       category,
       SUM(amount) AS total_spend
FROM transactions
WHERE customer_id IN (
    SELECT customer_id
    FROM customers
    WHERE tier = 'Premium'
)
GROUP BY customer_id, txn_month, category
ORDER BY customer_id, txn_month, total_spend DESC;
```

LinkedIn@AshishZope

This shows premium customer spending by category and month.

If you want, I can also turn this into:

1. a cleaner interview-ready PDF-style version,
2. a short 1-page revision,
3. or a more advanced version with real-world project-based SQL questions.

